



PROJECT DOCUMENTATION

Implement Client Script & UI Policy (Incident)

ServiceNow Incident Management

Team ID	6ab221341												
Team Members	<table> <tr> <th>Team Member</th><th>Role</th></tr> <tr> <td>Padmasri M</td><td>Team Lead & Project Coordinator</td></tr> <tr> <td>Vinitha K</td><td>ServiceNow UI Policy Developer</td></tr> <tr> <td>Vijaya M</td><td>ServiceNow Client Script Developer</td></tr> <tr> <td>Vijayalakshmi S</td><td>Testing & Quality Analyst</td></tr> <tr> <td>Yoga Azhagu H</td><td>Documentation & UI Analyst</td></tr> </table>	Team Member	Role	Padmasri M	Team Lead & Project Coordinator	Vinitha K	ServiceNow UI Policy Developer	Vijaya M	ServiceNow Client Script Developer	Vijayalakshmi S	Testing & Quality Analyst	Yoga Azhagu H	Documentation & UI Analyst
Team Member	Role												
Padmasri M	Team Lead & Project Coordinator												
Vinitha K	ServiceNow UI Policy Developer												
Vijaya M	ServiceNow Client Script Developer												
Vijayalakshmi S	Testing & Quality Analyst												
Yoga Azhagu H	Documentation & UI Analyst												
Institution / Department	Government Arts And Science College for Women, Sathankulam BSc. Computer Science												
Submission Date	28-09-2026												

Prepared in the requested project-documentation format. Replace bracketed fields and screenshot placeholders with your team information and actual evidence.

1. Introduction

This project demonstrates how ServiceNow Incident Management can be customized using Client Scripts and UI Policies to improve the behaviour of the Incident form. The configuration is intended to guide users, validate information, and control field behaviour while creating or updating incidents.

Project Title: Implement Client Script & UI Policy (Incident)

Team Members: [List team members and their roles]

2. Project Overview

Purpose

The purpose of this project is to implement client-side form behaviour in the ServiceNow Incident module using Client Scripts and UI Policies. The solution focuses on making the Incident form easier to use and improving consistency when users enter or modify incident information.

Features

- Controls Incident form fields based on selected values or form state.
- Supports field behaviours such as mandatory, visible/hidden, and read-only.
- Uses Client Script logic for dynamic validation or field updates.
- Uses UI Policy rules for declarative field behaviour.
- Improves consistency and reduces incorrect or incomplete Incident form entries.

Project Scope

The scope is limited to Incident form configuration and testing of Client Script and UI Policy behaviour in a ServiceNow instance. No custom MERN stack, external database, or separate web application is required for this implementation.

3. Architecture

Platform / User Interface

The user interface is the ServiceNow Incident form. Users interact with standard Incident fields, and the configured rules respond to form events and field values.

Client-Side Logic

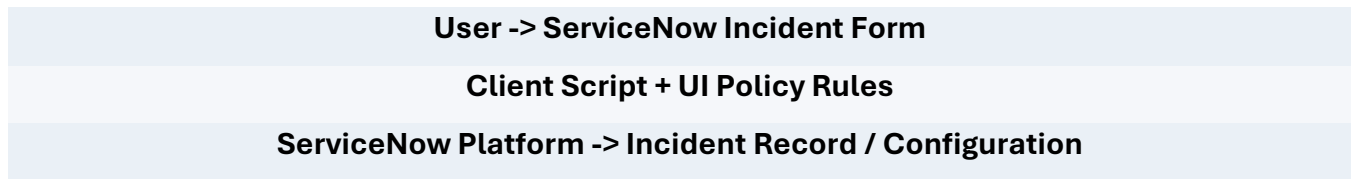
Client Scripts execute in the user interface and are used when custom JavaScript logic is needed for events such as form load, field change, or form submission. The exact Client Script type used in this project should match the event demonstrated by the team.

UI Policy Layer

UI Policies provide condition-based control of Incident form fields. UI Policy Actions specify what should happen to selected fields when the policy condition is true or false.

ServiceNow Data / Platform Layer

The Incident record and configuration records are managed by the ServiceNow platform. The project uses the platform database and security model rather than a separately deployed MongoDB or SQL database.



4. Setup Instructions

Prerequisites

- A ServiceNow developer or training instance with access to Incident Management.
- A user account with sufficient permissions to create or modify Client Scripts and UI Policies.
- A modern web browser such as Chrome or Edge.
- The project requirements and test cases prepared before configuration.

Installation / Configuration Steps

1. Sign in to the ServiceNow instance.
2. Open the Incident module and identify the form and fields used in the project.
3. Create or open the required UI Policy and enter the required condition.
4. Configure the related UI Policy Actions for the target fields.
5. Create or open the required Client Script and select the appropriate script type and table.
6. Add the JavaScript logic required by the project and save the record.
7. Open the Incident form and test the behaviour using different field values and form states.
8. Capture screenshots of the configuration and working output for the documentation.

Note: Exact field names, conditions, script names, and script code should be copied from the team's ServiceNow implementation so that the documentation matches the configured instance.

5. Folder Structure / Project Organization

ServiceNow configuration projects do not normally use a local client/server folder structure like a MERN application. The project is organized through configuration records inside the ServiceNow instance.

Component	Project Organization
Incident Table	Target table where the form behaviour is implemented.
UI Policy	Stores the condition used to control form field behaviour.
UI Policy Actions	Defines mandatory, visible, or read-only behaviour for selected fields.
Client Script	Stores client-side JavaScript logic for the Incident form.
Test Evidence	Screenshots, test cases, and observed results maintained for the report.
Demo Link	Project demonstration video or presentation link.

6. Running the Application / Demonstration

Because this is a ServiceNow configuration project, there is no npm start command. The application is demonstrated directly through the ServiceNow instance.

1. Log in to the ServiceNow instance.
2. Navigate to Incident Management and open a new or existing Incident record.
3. Enter or modify the field values required by the project scenario.
4. Observe how the UI Policy changes the selected field properties.
5. Trigger the Client Script event used in the project and verify the expected behaviour.
6. Save or submit the Incident and confirm that the record is handled as expected.

Demo Video: <https://youtu.be/zJ7xHkldHc0?si=59vywJQe7XMSdrUr>

7. API / Configuration Documentation

This project does not expose a custom REST API. The main implementation is based on ServiceNow configuration records and client-side form logic. The following table documents the configuration elements used by the solution.

Element	Purpose	Input / Trigger	Expected Behaviour
UI Policy	Applies conditional field behaviour.	[Condition used in project]	Target fields change according to the configured UI Policy Actions.
UI Policy Action	Controls a selected field property.	UI Policy condition	Field becomes mandatory, visible/hidden, or read-only as configured.
Client Script	Performs custom client-side logic.	[onLoad / onChange / onSubmit / other]	Configured JavaScript executes and updates or validates the form.
Incident Form	User interaction layer.	User enters or changes data.	The configured rules respond in real time or at submission.

8. Authentication

Authentication is provided by the ServiceNow platform. Users sign in to the instance using the credentials and authentication mechanism configured by the organization or training environment. Access to configuration records depends on the roles and permissions assigned to the user.

The project itself does not implement a separate JWT token, Express session, or custom login page.

9. User Interface

The main user interface is the ServiceNow Incident form. Screenshots should show both the configuration records and the visible result on the Incident form.

The screenshot shows the ServiceNow Incident form titled "Incident - Create INC0010010". The form is divided into two main sections: "Incident" and "New record". The "Incident" section contains fields for Number, Channel, State, Impact, Urgency, Priority, Assignment group, and Assigned to. The "New record" section contains fields for Caller, Category, Subcategory, Service, Service offering, Configuration item, Short description, and Description. The form is currently in a "New record" state, with the "Number" field highlighted in green. The "Priority" field is set to "4 - Low". The "Channel" field is set to "-- None --". The "State" field is set to "New". The "Impact" field is set to "3 - Low". The "Urgency" field is set to "3 - Low". The "Assignment group" field is empty. The "Assigned to" field is empty. The "Short description" field is empty. The "Description" field is empty. The form has "Submit" and "Resolve" buttons at the top right. A "Related Search Results" button is at the bottom center. A "Related Search" field is at the bottom left.

Figure 1. Incident form before Client Script and UI Policy configuration

The screenshot shows the ServiceNow UI Policy configuration screen titled "Client Script - EF - Show Config Help Info". The form is divided into two main sections: "Client Script" and "EF - Show Config Help Info". The "Client Script" section contains fields for Name, Table, UI Type, and Type. The "EF - Show Config Help Info" section contains fields for Application, Active, Inherited, and Global. The form is currently in a "Client Script" state, with the "Name" field highlighted in green. The "Table" field is set to "Error Config [sys_error_config]". The "UI Type" field is set to "All". The "Type" field is set to "onLoad". The "Application" field is set to "Global". The "Active" checkbox is checked. The "Inherited" checkbox is unchecked. The "Global" checkbox is checked. The "Description" field is empty. The "Messages" field is empty. The "Script" field contains the following code:

```
1 function onLoad() {
2   var gwt = new GwtMessage();
3   var message = gwt.getMessage('Click on "Configuration Help" for more information on configuration parameters');
4   g_form.addInfoMessage(message);
5 }
6
```

Figure 2. UI Policy configuration screen

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: High Impact Control

Table: Incident [Incident]

* Field name: Click to select...

Application: Global

Mandatory: True

Visible: True

Read only: Leave alone

Clear the field value: ☐

Submit

Figure 3. UI Policy Actions for the selected Incident fie

Client Script - EF - Auto-populate Severity Label

Name: EF - Auto-populate Severity Label

Table: Error Code [sys_error_code]

UI Type: All

Type: onLoad

Application: Global

Active: ☒

Inherited: ☐

Global: ☒

Description: Sets the initial severity label on form load based on the default severity value

Messages:

Script

```

1 function onLoad() {
2     var severityValue = parseInt(g_form.getValue('severity'), 10);
3     var severityLabel = valueToLabel(severityValue);
4     if (severityLabel)
5         g_form.setValue('severity_label', severityLabel);
6
7     /**
8      * Converts a numeric severity value to its label
9      * @param {number} value - The numeric severity value
10     * @returns {string|null} The severity label or null if invalid
11     */
12     function valueToLabel(value) {
13         if (typeof value !== 'number' || isNaN(value) || value < 0 || value > 499) {
14             return null;
15         }
16     }

```

Figure 4. Client Script configuration screen

10. Testing

Testing is performed by opening the Incident form and checking the configured behaviour under the required conditions. Functional testing should confirm that the correct fields change state and that the Client Script executes for its selected trigger.

Test ID	Test Scenario	Expected Result	Observed Result
TC-01	Open the Incident form.	Form loads without configuration errors.	[Enter observed result]
TC-02	Set the condition that activates the UI Policy.	Configured target fields change according to the UI Policy Actions.	[Enter observed result]
TC-03	Change the field used by the Client Script.	Client Script logic runs and applies the intended field/value change.	[Enter observed result]
TC-04	Enter an invalid or incomplete value, where validation is configured.	The form shows the intended validation message or prevents the invalid action.	[Enter observed result]
TC-05	Return the form to a non-triggering condition.	Fields return to the configured normal behaviour.	[Enter observed result]
TC-06	Save / submit the Incident.	Incident is saved or submitted according to the configured rules.	[Enter observed result]

Testing tools: ServiceNow Incident form, browser developer tools (when needed), and the team's prepared test-case sheet.

11. Screenshots or Demo

Insert the final screenshots captured from the team's ServiceNow instance. Each screenshot should clearly show the relevant configuration or working result.

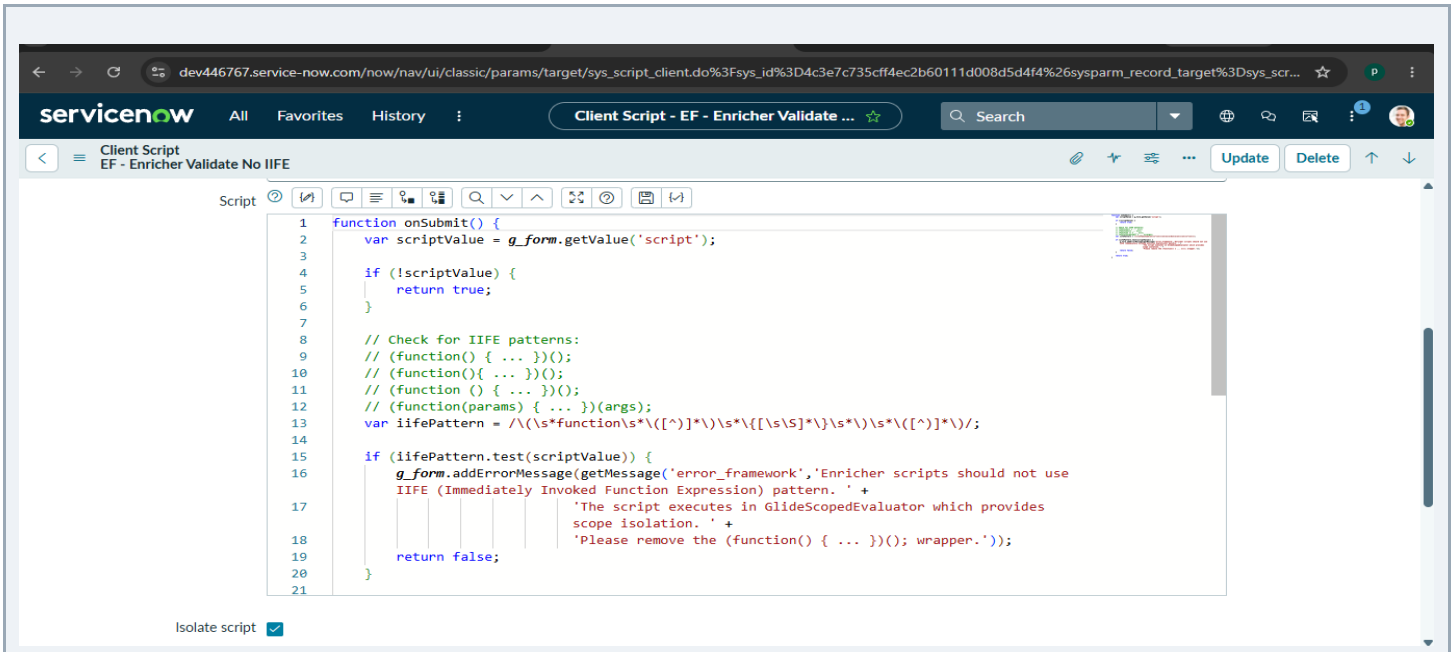


Figure 5. UI Policy working result on the Incident form

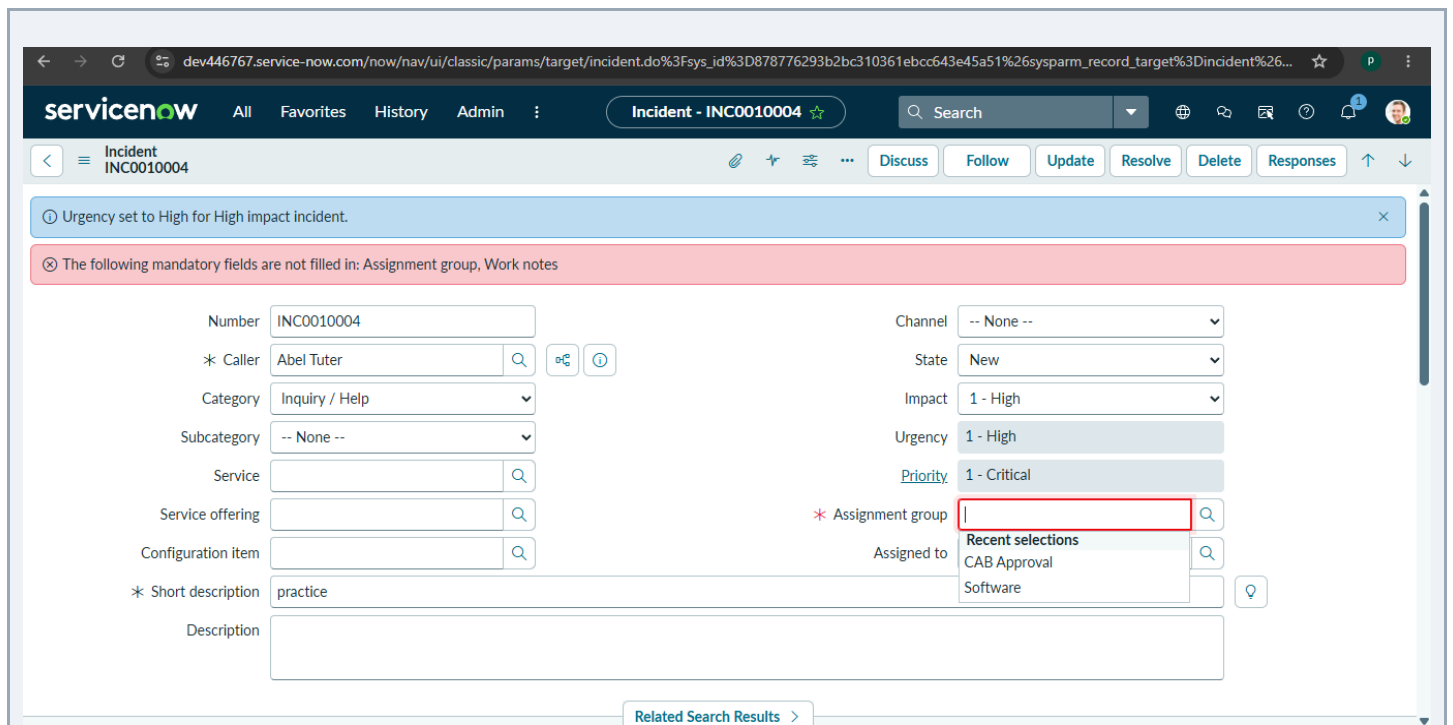


Figure 6. Client Script working result on the Incident form

The screenshot shows the ServiceNow Incident form for incident INC0010006. The form is titled "Incident - INC0010006" and includes a search bar and navigation buttons (Discuss, Follow, Update, Resolve, Delete, Responses). The form fields are organized into two columns:

- Left Column:**
 - Number: INC0010006
 - * Caller: Amelia Caputo
 - Category: Inquiry / Help
 - Subcategory: -- None --
 - Service:
 - Service offering:
 - Configuration item:
 - * Short description: testing UI policy
 - Description:
- Right Column:**
 - Channel: -- None --
 - State: In Progress
 - Impact: 1 - High
 - Urgency: 1 - High
 - Priority: 1 - Critical
 - * Assignment group: Software
 - Assigned to: Beth Anglin

At the bottom of the form, there is a button labeled "Related Search Results >".

Figure 7. Incident successfully created / updated after testing

Demo Link: https://drive.google.com/file/d/1hTul6jo_Weyynf-S3-442fkkJAEhS_rp/view?usp=drivesdk

12. Known Issues

- Client-side behaviour depends on the selected table, form view, script type, and field names.
 - Incorrect conditions or field names may cause the intended UI behaviour not to appear.
 - Configuration changes may require saving, refreshing, or reopening the Incident form before the result is visible.
 - User permissions and instance-specific settings can affect access to configuration records.
- Before submission, replace this section with any real issue found during your team testing. Remove items that are not applicable to your instance.

13. Future Enhancements

- Add more Client Scripts for advanced Incident form validation and automation.
- Create additional UI Policies for different Incident states, priorities, categories, or assignment scenarios.
- Introduce automated testing for repeated Incident form scenarios.
- Extend the solution with Flow Designer, Business Rules, Notifications, or ServiceNow Workspace improvements where required.
- Package the completed configuration in a controlled update set or application for easier migration between instances.

Appendix

A. Source / Configuration Evidence

- Client Script record: [Enter script name / sys_id]
- UI Policy record: [Enter policy name / sys_id]
- UI Policy Actions: [Enter target field names]
- Incident table: [Enter table name, e.g., incident]

B. Source Code

Paste the exact Client Script code used in the project below. Do not change variable names or logic when documenting the final implementation.

```
function onLoad() {  
  
    var portfolioId = g_form.getValue("spm_service_portfolio");  
    var layerId = g_form.getValue("spm_taxonomy_layer");  
  
    var ga = new GlideAjax("global.SPMClientUtilAjax");  
    ga.addParam("sysparm_name", "getLeafOrder");  
    ga.addParam("sysparm_portfolio", portfolioId);  
    ga.addParam("sysparm_layer", layerId);  
    ga.getXML(renderRelatedList);  
  
    function renderRelatedList(response) {  
        var answer = response.responseXML.documentElement.getAttribute("answer");  
        if(answer && answer == 'leaf') {  
            g_form.showRelatedList('cmdb_ci_service.spm_taxonomy_node');  
        } else {  
            g_form.hideRelatedList('cmdb_ci_service.spm_taxonomy_node');  
        }  
    }  
}
```

C. Dataset Link

Not applicable - the project uses ServiceNow Incident records and configuration rather than a separate training dataset.

D. GitHub & Demo Link

GitHub: <https://github.com/padmasrim071-bot/Implement-Client-Script-UI-policy-Incident->

Demo: https://drive.google.com/file/d/1hTul6jo_Weyynf-S3-442fkkJAEhS_rp/view?usp=drivesdk